

02 Final Result Report

Final outputs, architecture summary, setup guide, API evidence, limitations, and improvements.

Repository	https://github.com/aalanani2000/al-rouf-ai-integration-assessment
Default branch	main
Latest commit	7dcd3eadb125137db59fb128b337ba47cc1d1dbb
Prepared date	2026-06-24
Assessment	AI Integration Engineer task pack

Final Outcome

The assessment implementation is complete across all three requested tasks. The repository contains runnable source code, Docker packaging, tests where required, exported workflow artifacts, setup documentation, screenshots, and the final reports.

Task	Implementation	Validation	Status
Task 1	n8n RFQ webhook, OpenAI extraction, mock CRM, bilingual reply, alert log.	Successful n8n executions, persisted JSON records, screenshot evidence.	Complete; attachment bytes scoped as future enhancement.
Task 2	FastAPI quotation API with catalog, pricing tiers, SQLite, Docker, OpenAPI.	Automated pytest suite and OpenAPI screenshot evidence.	Complete.
Task 3	FastAPI bilingual RAG service over English/Arabic documents with citations and refusal.	Unit tests, integration test path, Arabic question evidence, error-handling screenshot.	Complete.

Task 1 Final Result

- Inbound RFQ data enters through an n8n webhook.
- OpenAI extracts urgency, key requirements, and a summary from the free-text message.
- The workflow creates a mock CRM record in `data/mock\_crm.json`.
- The workflow drafts English and Arabic reply text without including pricing.
- The workflow creates an internal sales alert in `data/alerts.json`.

Task 2 Final Result

- FastAPI service exposes health, catalog, quote creation, and quote retrieval endpoints.
- Bulk discount tiers are applied by total quote quantity.
- SQLite stores generated quotes and survives restarts when Docker volume/data mapping is used.

- OpenAPI documentation is available at `/docs` while the service is running.
- Automated tests cover expected success and failure paths.

Task 3 Final Result

- The service indexes 4 sample documents covering product specifications, warranty policy, and shipping terms.
- English and Arabic questions can retrieve relevant source chunks.
- Responses cite source documents and include latency and estimated cost metadata.
- Out-of-scope questions are refused when retrieval confidence is insufficient.
- Embeddings are cached locally to avoid unnecessary repeated embedding calls.

Architecture Explanation

- Task 1 uses n8n for orchestration because the workflow itself can be exported, versioned, reviewed, and rerun locally.
- Task 2 separates API schemas, catalog data, pricing logic, database access, and route handlers for maintainability.
- Task 3 keeps the RAG system intentionally lightweight because the assessment document set is small; a full vector database would be unnecessary infrastructure at this scale.

Run And Test Commands

Area	Command	Purpose
Task 1	<code>docker-compose up -d</code>	Start n8n from the repository root.
Task 2	<code>cd task2_quotation_microservice && docker-compose up --build</code>	Run quotation API on port 8000.
Task 2 tests	<code>cd task2_quotation_microservice && PYTHONPATH=. pytest tests/ -v</code>	Run quotation service tests.
Task 3	<code>cd task3_bilingual_rag && docker-compose up --build</code>	Run bilingual RAG API on port 8001.
Task 3 tests	<code>cd task3_bilingual_rag && PYTHONPATH=. pytest tests/ -v</code>	Run RAG tests; API-key integration tests skip when unset.

Repository Guidance

Use this structure map to navigate the solution bundle and GitHub repository:

```
C:.\
|  .env
|  .env.example
|  .gitignore
|  docker-compose.yml
|  README.md
|
+---data
|     alerts.json
|     mock_crm.json
```

```

|
+---docs
|   +---architecture
|   +---evidence
|       \---task1_rfqcrm_automation
|           execution_notes.md
|       \---screenshots
|           +---Task 1
|               javascript code 1 error .png
|               n8n canva.png
|               open ai node.png
|               proof of the volumes being correctly attached.png
|               repository structure.png
|               set field output.png
|               set field.png
|               Webhook node.png
|               workflow progress .png
|               workflow progress 1 .png
|               workflow succes.png
|               workflow.png
|               workflow2.png
|           +---Task 2
|               Open API.png
|           \---Task 3
|               a problem when communicating with model.png
|               testing the RAG on arabic question.pdf
|
+---reports
|   +---01_execution_evidence_report
|   \---02_final_result_report
+---task1_rfqcrm_automation
|   \---n8n_workflows
|       rfqc-crm-automation.json
|
+---task2_quotation_microservice
|   .gitignore
|   docker-compose.yml
|   Dockerfile
|   README.md
|   requirements.txt
|
|   +---app
|       catalog.py
|       database.py
|       main.py
|       quoting.py
|       schemas.py
|       __init__.py
|
|   +---data
|       quotes.db
|
|   \---tests
|       test_quotation.py
|       __init__.py
|
\---task3_bilingual_rag
|   .env
|   .env.example
|   .gitignore
|   docker-compose.yml
|   Dockerfile
|   README.md
|   requirements.txt
|
|   +---app
|       embeddings.py
|       llm.py
|       loader.py
|       main.py

```

```
|      schemas.py
|      __init__.py
|
+---data
|      embeddings_cache.json
|
+---documents
|      doc1_product_spec_high_bay_en.md
|      doc2_warranty_policy_ar.md
|      doc3_shipping_terms_en.md
|      doc4_product_spec_street_light_ar.md
|
\---tests
      test_rag.py
      __init__.py
```

Error Handling, Maintainability, And Security

- Task 1 persists n8n state and mock CRM data with Docker volumes and local bind mounts.
- Task 2 validates input with Pydantic, returns clear HTTP errors, and avoids leaking internal stack traces to clients.
- Task 3 refuses unsupported questions through retrieval thresholding rather than relying only on prompt instructions.
- Secrets are stored in `.env` files or runtime environment variables and are not hardcoded in source files.
- The FastAPI codebases keep core logic separate from HTTP handlers so behavior can be tested directly.

Future Enhancements

- Task 1: ingest RFQs from real email inboxes, WhatsApp Business, website forms, or CRM webhooks instead of only a simulated webhook.
- Task 1: archive actual attachment files under each CRM record and add duplicate detection.
- Task 2: add a quotation UI, quote PDF generation, authentication, configurable product/pricing administration, taxes, shipping, and ERP/CRM integration.
- Task 3: add a polished bilingual chat UI, document upload and re-indexing, richer citation snippets, streaming responses, access control, and a vector database if the corpus grows.

AI Assistance Disclosure

AI assistance was used for brainstorming architecture choices, debugging explanations, starter snippets, and documentation/report polish. The candidate remained responsible for all local execution, implementation, testing, configuration, screenshots, trade-off selection, and final submission packaging.